



1928

K. N. Toosi University of Technology

Bank Customer Churn Analysis

Predictive Modeling using Neural Networks

Student Name: Mohammad Hossein Bakhshian

Student ID: 40215443

Course: Introduction to Artificial Intelligence

Instructor: Dr.Mahdi Aliyari Shooredeli

August 12, 2026

Contents

1	Introduction and Project Objectives	2
2	Phase 1: Initial Data Exploration	2
2.1	Python Implementation: Loading Data	2
2.2	Dataset Structure Analysis	3
2.2.1	Initial Observations	4
2.3	Data Cleaning and Descriptive Statistics	4
2.3.1	Statistical Insights	5
3	Exploratory Data Analysis (EDA)	5
3.1	Visualization Insights	6
4	Data Preprocessing	7
5	Multilayer Perceptron (MLP) Modeling and Batch Strategy Analysis	9
5.1	Network Architecture Design	9
5.2	Implementation details and Python Script	9
5.3	Empirical Analysis of Convergence and Computational Dynamics	12
6	Model Evaluation and Performance Metrics	12
6.1	Evaluation Implementation	13
6.2	Analysis of the Results	14
7	Conclusion	14

1 Introduction and Project Objectives

Customer churn prediction is a critical task for financial institutions aiming to maximize customer lifetime value. This project focuses on analyzing customer behavior within a banking environment to predict the likelihood of a customer leaving the bank. By proactively identifying at-risk customers, banks can implement targeted retention strategies.

As proposed in the initial roadmap, this study follows a multi-phase approach:

1. **Phase 1: Data Preprocessing and EDA:** Rigorous cleaning, encoding of categorical variables, and exploratory data analysis to prepare the dataset for machine learning and deep learning algorithms.
2. **Phase 2: MLP Neural Networks:** Implementing and tuning a Multilayer Perceptron (MLP) to model the complex, non-linear relationships inherent in customer churn behavior.

2 Phase 1: Initial Data Exploration

The success of any Artificial Intelligence model depends heavily on the quality, structure, and understanding of the underlying data. In this initial step, we focus on environment setup, data ingestion from local storage, and structural validation to ensure data integrity before advancing to exploratory data analysis (EDA).

2.1 Python Implementation: Loading Data

We utilize the `pandas` library, the standard for data manipulation in Python, alongside `numpy` for numerical operations. The following code demonstrates the ingestion of the dataset from the local environment and the initial inspection of the upper boundary (first few rows) of the data frame.

```

1      # Step 1: Import Libraries and Load the Dataset
2      import numpy as np
3      import pandas as pd
4
5      # Load the bank churn dataset
6      df = pd.read_csv(r"C:\Users\Asus\Desktop\S6\
7                      Primitive_AI_star\My_Final_Project\Churn.csv")
8
9      # Display the first 5 rows to understand the structure of
      our data
      df.head()
```

Listing 1: Loading dataset and displaying top rows

Listing 2: Output of `df.head()`

RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	
	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited		
0	1	15634602	Hargrave	619	France	Female	42	2
	0.00	1	1	1	101348.88			
1	2	15647311	Hill	608	Spain	Female	41	1
	83807.86	1	0	1	112542.58			
0								

2	3	15619304	Onio	502	France	Female	42	8
	159660.80		3	1	0		113931.57	1
3	4	15701354	Boni	699	France	Female	39	1
	0.00		2	0	0		93826.63	
0								
4	5	15737888	Mitchell	850	Spain	Female	43	2
	125510.82		1	1	1		79084.10	0

Examining the first few rows confirms that the CSV file has been parsed correctly and the headers align accurately with the columns. To ensure consistency across the entire dataset, we also inspect the lower boundary (last few rows).

```
1 # Display the last 5 rows to understand the structure of
2   our data
   df.tail()
```

Listing 3: Displaying bottom rows

Listing 4: Output of df.tail()

RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure
Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited		
9995	9996	15606229	Obijiaku	771	France	Male	39
	5	0.00	2	1		0	
	96270.64	0					
9996	9997	15569892	Johnstone	516	France	Male	35
10	57369.61		1	1	1		101699.77
0							
9997	9998	15584532	Liu	709	France	Female	36
	7	0.00	1	0		1	
	42085.58	1					
9998	9999	15682355	Sabbatini	772	Germany	Male	42
	3	75075.31	2	1		0	
	92888.52	1					
9999	10000	15628319	Walker	792	France	Female	28
	4	130142.79	1	1		0	
	38190.78	0					

2.2 Dataset Structure Analysis

Before proceeding with feature engineering or visualization, a technical overview of the dataset's metadata is required. We extract the column data types, non-null counts, and memory footprint using the `info()` method. This step is crucial for identifying missing values that would otherwise cause model failure.

```
1 # display the basic information of our data types and
2   missing values
   df.info()
```

Listing 5: Checking dataframe information

Listing 6: Output of df.info()

<class 'pandas.core.frame.DataFrame'>				
RangeIndex: 10000 entries, 0 to 9999				
Data columns (total 14 columns):				
#	Column	Non-Null	Count	Dtype
---	-----	-----	-----	-----
0	RowNumber	10000	non-null	int64
1	CustomerId	10000	non-null	int64
2	Surname	10000	non-null	object
3	CreditScore	10000	non-null	int64
4	Geography	10000	non-null	object
5	Gender	10000	non-null	object

```

6   Age          10000 non-null  int64
7   Tenure       10000 non-null  int64
8   Balance     10000 non-null  float64
9   NumOfProducts 10000 non-null  int64
10  HasCrCard    10000 non-null  int64
11  IsActiveMember 10000 non-null  int64
12  EstimatedSalary 10000 non-null float64
13  Exited       10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB

```

2.2.1 Initial Observations

Based on the console output above, several key insights are drawn which will guide our preprocessing strategy:

- **Data Integrity:** The dataset is robust regarding completeness. All 14 columns contain exactly 10,000 non-null entries.
- **Feature Types:** The independent variables consist of a mixture of continuous numerical features (`float64`, `int64`) and categorical features (`object`). The categorical features (`Geography`, `Gender`) will require encoding (e.g., One-Hot Encoding) before feeding them into the MLP neural network.
- **Redundant Variables:** The variables `RowNumber`, `CustomerId`, and `Surname` act merely as unique identifiers and carry no predictive signal for churn behavior. Including them would introduce noise to the model; thus, they will be discarded in the subsequent data cleaning phase.

2.3 Data Cleaning and Descriptive Statistics

Following our initial data loading, we perform essential cleaning operations. The primary steps include removing columns that are irrelevant for predictive modeling, such as unique identifiers (`RowNumber`, `CustomerId`, `Surname`), and eliminating any columns that are entirely empty, which can arise from file import artifacts. Subsequently, we generate statistical summaries to understand the distribution, central tendency, and dispersion of the numerical features.

```

1 # Step 2: Data Cleaning and Descriptive Statistics
2
3 # 1. Dropping specific redundant columns safely
4 # Using errors='ignore' prevents KeyError if the cell is executed
   multiple times
5 columns_to_drop = ["RowNumber", "CustomerId", "Surname"]
6 df = df.drop(columns=columns_to_drop, errors="ignore")
7
8 # 2. Dropping completely empty columns (e.g., 'Unnamed: 14')
9 # The parameter 'how='all'' ensures a column is dropped only if
   all its values are NaN
10 df = df.dropna(axis=1, how='all')
11
12 # 3. Generating descriptive statistics for numerical features
13 # The output is transposed (.T) for better readability and
   captured in Table \ref{tab:desc_stats}.

```

```
14 df.describe().T
```

Listing 7: Data Cleaning and Statistical Summary Generation

The output of the final command, which provides a statistical overview of the data, is not shown as a raw console output. Instead, it is formatted into Table 1 for enhanced clarity and professional presentation.

Table 1: Descriptive Statistics of Numerical Features

Feature	count	mean	std	min	25%	50%	75%	max
CreditScore	10000.0	650.528	96.653	350.00	584.00	652.000	718.000	850.00
Age	10000.0	38.921	10.487	18.00	32.00	37.000	44.000	92.00
Tenure	10000.0	5.012	2.892	0.00	3.00	5.000	7.000	10.00
Balance	10000.0	76485.8	62397.4	0.00	0.00	97198.5	127644.	250898.
NumOfProducts	10000.0	1.530	0.581	1.00	1.00	1.000	2.000	4.00
HasCrCard	10000.0	0.705	0.455	0.00	0.00	1.000	1.000	1.00
IsActiveMember	10000.0	0.515	0.499	0.00	0.00	1.000	1.000	1.00
EstimatedSalary	10000.0	100090.	57510.4	11.58	51002.1	100193.	149388.	199992.
Exited	10000.0	0.2037	0.402	0.00	0.00	0.000	0.000	1.00

2.3.1 Statistical Insights

From Table 1, we deduce several key points:

- **Target Variable (Exited):** The mean is approximately 0.2037, indicating that $\approx 20.37\%$ of the customers in this dataset have churned. This highlights an imbalanced dataset, which is a critical consideration for model training and evaluation.
- **Age:** The customer base is diverse, with ages ranging from 18 to 92 years. The central tendency is around 37-39 years (median and mean), suggesting a middle-aged customer profile.
- **Balance:** A significant observation is that the 25th percentile for **Balance** is 0.00. This implies that more than a quarter of the customers have a zero balance, which could be a strong predictor of churn.

3 Exploratory Data Analysis (EDA)

In this section, we conduct an Exploratory Data Analysis (EDA) to better understand the underlying patterns and relationships within the dataset. The primary objectives are to evaluate the distribution of the target variable (customer churn) and to assess the linear correlation among the numerical features.

The following Python script utilizes the **seaborn** and **matplotlib** libraries to generate a frequency distribution of the **Exited** variable and a correlation heatmap.

```
1 # Step 3: Exploratory Data Analysis (EDA)
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
```

```

5 # Set a clean and professional visual style
6 sns.set_theme(style="whitegrid", context="paper", font_scale=1.2)
7
8 # Create a figure with two subplots side by side
9 fig, axes = plt.subplots(1, 2, figsize=(16, 6))
10
11 # --- Plot 1: Distribution of Churn (Exited) ---
12 # Added hue='Exited' and legend=False to fix seaborn
    FutureWarning
13 sns.countplot(data=df, x='Exited', hue='Exited', ax=axes[0],
14 palette='pastel', legend=False, edgecolor='black')
15 axes[0].set_title("Distribution of Customer Churn (Exited)",
    fontsize=14, fontweight='bold', pad=15)
16 axes[0].set_xlabel('Exited Status (0 = Retained, 1 = Churned)',
    fontsize=12)
17 axes[0].set_ylabel('Number of Customers', fontsize=12)
18
19 # Add value counts on top of bars for better readability
20 for p in axes[0].patches:
21     axes[0].annotate(f'{int(p.get_height()):,}',
22 (p.get_x() + p.get_width() / 2., p.get_height()),
23 ha='center', va='bottom', fontsize=11, fontweight='bold',
24 xytext=(0, 5), textcoords='offset points')
25
26 # --- Plot 2: Correlation Heatmap of Numerical Features ---
27 # Select only numerical columns for correlation
28 numerical_cols = df.select_dtypes(include=['int64', 'float64']).
    columns
29 corr_matrix = df[numerical_cols].corr()
30
31 # Draw the heatmap with a professional color map (cmap)
32 sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='RdYlBu_r',
33 square=True, ax=axes[1], cbar_kws={"shrink": .8}, linewidths=0.5)
34 axes[1].set_title('Correlation Matrix of Numerical Features',
    fontsize=14, fontweight='bold', pad=15)
35
36 # Adjust layout and save the plot
37 plt.tight_layout()
38 plt.savefig('eda_plots.png', dpi=300, bbox_inches='tight')
39 plt.show()

```

Listing 8: Generating EDA Visualizations

3.1 Visualization Insights

As illustrated in Figure 1, the visual analysis yields the following critical insights:

- **Class Imbalance:** The left subplot demonstrates that only a minority of the customers (approximately 20%) have exited the bank (*Exited* = 1). This imbalanced nature of the dataset dictates that relying solely on Accuracy for our upcoming

Multilayer Perceptron (MLP) model evaluation will be misleading. Metrics such as F1-Score, Precision, and Recall must be prioritized.

- **Correlation Matrix:** The heatmap on the right indicates that **Age** has the highest positive correlation (approximately 0.29) with the target variable, suggesting that older customers are more likely to churn. Furthermore, the independent variables show minimal correlation with each other, which favorably reduces the risk of multicollinearity in our predictive models.

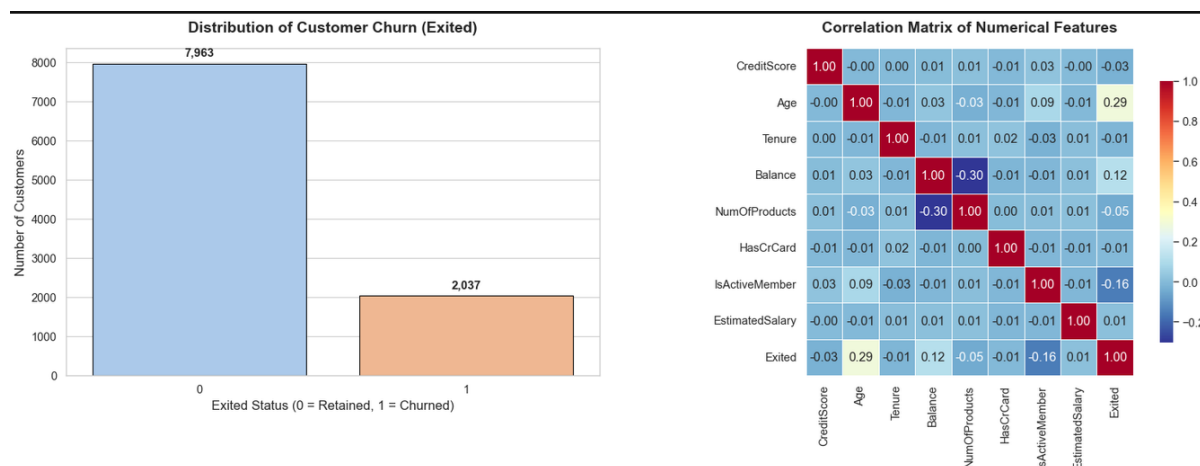


Figure 1: Frequency distribution of the target variable (left) and the correlation matrix of numerical features (right).

4 Data Preprocessing

Data preprocessing is a critical step before training any Artificial Neural Network (ANN). Multilayer Perceptrons (MLPs) are highly sensitive to the scale and format of the input data. Therefore, the following transformations were applied to the dataset:

- **Categorical Encoding:** The categorical features **Geography** and **Gender** were transformed into numerical formats using One-Hot Encoding. To prevent multicollinearity (the dummy variable trap), the first category was dropped using `drop_first=True`.
- **Train-Test Split:** The dataset was partitioned into a training set (80%) and a testing set (20%). The split was stratified based on the target variable (**Exited**) to preserve the original class distribution in both subsets, which is crucial given our imbalanced dataset.
- **Feature Scaling:** Standard scaling (Z-score normalization) was applied to the numerical features. The scaler was fitted strictly on the training set to prevent data leakage, and then used to transform both the training and testing sets.

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler
4
5 # Step 4: Data Preprocessing
6
7 # 1. Encoding categorical variables (Geography and Gender)
8 # We use drop_first=True to avoid the dummy variable trap (
9     multicollinearity)
10 df_encoded = pd.get_dummies(df, columns=['Geography', 'Gender'],
11     drop_first=True)
12
13 # 2. Separating Features (X) and Target (y)
14 X = df_encoded.drop('Exited', axis=1)
15 y = df_encoded['Exited']
16
17 # 3. Train-Test Split (80% Train, 20% Test)
18 # stratify=y ensures that the proportion of churned/retained
19     customers is maintained in both sets
20 X_train, X_test, y_train, y_test = train_test_split(X, y,
21     test_size=0.2, random_state=42, stratify=y)
22
23 # 4. Feature Scaling (Standardization)
24 # MLPs are sensitive to feature scaling, so we scale features to
25     have mean=0 and variance=1
26 scaler = StandardScaler()
27
28 # Fit on training data and transform it
29 X_train_scaled = scaler.fit_transform(X_train)
30 # Only transform the test data (do NOT fit) to prevent data
31     leakage
32 X_test_scaled = scaler.transform(X_test)
33
34 # Display the shape of our final datasets
35 print("Data Preprocessing Completed Successfully!")
36 print("-" * 40)
37 print(f"Training features (X_train_scaled) shape: {X_train_scaled
38     .shape}")
39 print(f"Testing features (X_test_scaled) shape: {X_test_scaled.
40     shape}")
41 print(f"Training target (y_train) shape: {y_train.shape}
42     ")
43 print(f"Testing target (y_test) shape: {y_test.shape}"
44     )
```

Listing 9: Data Preprocessing: Encoding, Splitting, and Scaling

Console Output: Preprocessing Result

Data Preprocessing Completed Successfully!

```
-----
Training features (X_train_scaled) shape: (8000, 11)
Testing features (X_test_scaled) shape: (2000, 11)
Training target (y_train) shape: (8000,)
Testing target (y_test) shape: (2000,)
```

5 Multilayer Perceptron (MLP) Modeling and Batch Strategy Analysis

In this section, we design and implement a Multilayer Perceptron (MLP) architecture to predict customer churn. An essential part of this phase involves comparing Full-Batch and Mini-Batch Gradient Descent methods to effectively analyze the models learning process.

5.1 Network Architecture Design

Given the preprocessed input feature space dimension ($N = 11$), a lightweight architecture was deliberately chosen to mitigate the risk of overfitting while maintaining sufficient representational capacity. The neural network comprises the following layers:

- **Input Layer:** Explicitly defined using the `Input(shape=(11,))` object to strictly specify input dimensions and prevent shape-related warnings.
- **Hidden Layers:** Two hidden layers containing 16 and 8 neurons, respectively. Both utilize the Rectified Linear Unit (ReLU) activation function to introduce non-linearity and avoid the vanishing gradient problem.
- **Output Layer:** A single neuron equipped with a Sigmoid activation function, mapping the output to a probability distribution $P(y = 1|X) \in [0, 1]$ suitable for our binary classification task (Churn vs. Retained).

The model is optimized using Stochastic Gradient Descent (SGD) with a learning rate of $\eta = 0.01$ and the `binary_crossentropy` loss function.

5.2 Implementation details and Python Script

To rigorously analyze the computational efficiency and convergence behavior, the identical architecture was trained using two distinct batching strategies over 100 epochs. The complete implementation, including model initialization, training loops, time tracking, and advanced visualization, is provided in Listing 11.

```
1 import os
2 os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
3
4 import tensorflow as tf
5 from tensorflow.keras.models import Sequential
```

```
6 from tensorflow.keras.layers import Dense, Input
7 import matplotlib.pyplot as plt
8 import time
9
10 # Setting random seed for reproducibility
11 tf.random.set_seed(42)
12
13 # Function to create our base MLP model architecture (Fixed
    Warning)
14 def create_model():
15     model = Sequential([
16         # Explicit Input layer for newer Keras versions to avoid warnings
17         Input(shape=(X_train_scaled.shape[1],), name='Input_Layer'),
18         Dense(16, activation='relu', name='Hidden_Layer_1'),
19         Dense(8, activation='relu', name='Hidden_Layer_2'),
20         Dense(1, activation='sigmoid', name='Output_Layer')
21     ])
22     model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate
        =0.01),
23                 loss='binary_crossentropy',
24                 metrics=['accuracy'])
25     return model
26
27 print("Initializing Neural Network Models...")
28
29 # -----
30 # 1. Mini-Batch Gradient Descent Training
31 # -----
32 print("Training with Mini-Batch Gradient Descent (Batch Size =
    32)...")
33 model_mini = create_model()
34 start_time_mini = time.time()
35
36 history_mini = model_mini.fit(X_train_scaled, y_train,
37                               epochs=100,
38                               batch_size=32,
39                               validation_split=0.2,
40                               verbose=0)
41 time_mini = time.time() - start_time_mini
42
43 # -----
44 # 2. Full-Batch Gradient Descent Training
45 # -----
46 print("Training with Full-Batch Gradient Descent (Batch Size =
    Full Dataset)...")
47 model_full = create_model()
48 start_time_full = time.time()
49
50 # Dynamically using the exact size of the training set for full
    batch
51 full_batch_size = X_train_scaled.shape[0]
```

```
52 history_full = model_full.fit(X_train_scaled, y_train,
53 epochs=100,
54 batch_size=full_batch_size,
55 validation_split=0.2,
56 verbose=0)
57 time_full = time.time() - start_time_full
58
59 print("-" * 50)
60 print(f"Mini-Batch Training Time: {time_mini:.2f} seconds")
61 print(f"Full-Batch Training Time: {time_full:.2f} seconds")
62
63 # -----
64 # 3. Professional and Beautiful Loss Curve Plot
65 # -----
66 fig, ax = plt.subplots(figsize=(10, 6), dpi=150) # Higher DPI for
        better resolution
67
68 # Setting a beautiful background
69 ax.set_facecolor('#f8f9fa')
70 fig.patch.set_facecolor('white')
71
72 # Plotting the lines with distinct colors and line styles
73 ax.plot(history_mini.history['loss'], label='Mini-Batch Loss (
        Train)', color='#005b96', linewidth=2.5)
74 ax.plot(history_full.history['loss'], label='Full-Batch Loss (
        Train)', color='#d9534f', linewidth=2.5, linestyle='--')
75
76 # Adding Titles and Labels with elegant fonts
77 ax.set_title('Convergence Speed: Mini-Batch vs. Full-Batch
        Gradient Descent', fontsize=16, fontweight='bold', pad=15)
78 ax.set_xlabel('Epochs', fontsize=13, fontweight='medium')
79 ax.set_ylabel('Binary Crossentropy Loss', fontsize=13, fontweight
        ='medium')
80
81 # Enhancing the Grid
82 ax.grid(color='white', linestyle='-', linewidth=1.5)
83
84 # Adding the Legend with a nice shadow and frame
85 legend = ax.legend(loc='upper right', fontsize=12, frameon=True,
        shadow=True, borderpad=1)
86 legend.get_frame().set_facecolor('white')
87
88 # Adding an annotation box inside the plot to show the training
        times
89 time_text = f"Wall-clock Times:\nMini-Batch: {time_mini:.1f} sec\
        \nFull-Batch: {time_full:.1f} sec"
90 ax.text(0.35, 0.65, time_text, transform=ax.transAxes, fontsize
        =12,
91 verticalalignment='center', horizontalalignment='center',
92 bbox=dict(facecolor='white', edgecolor='#cccccc', boxstyle='round
        ,pad=0.8', alpha=0.9))
```

```

93 plt.tight_layout()
94 # Saving the enhanced plot with a new name
95 plt.savefig('loss_comparison_enhanced.png', dpi=300, bbox_inches=
96             'tight')
97 plt.show()

```

Listing 10: Complete implementation of the MLP model, Training Strategies, and Visualization

5.3 Empirical Analysis of Convergence and Computational Dynamics

Figure 2 vividly illustrates the trajectory of the binary crossentropy loss for both strategies over the 100-epoch training phase. As hypothesized, the Mini-Batch strategy (batch size 32) exhibits a remarkably steeper descent during the initial epochs compared to the Full-Batch approach.

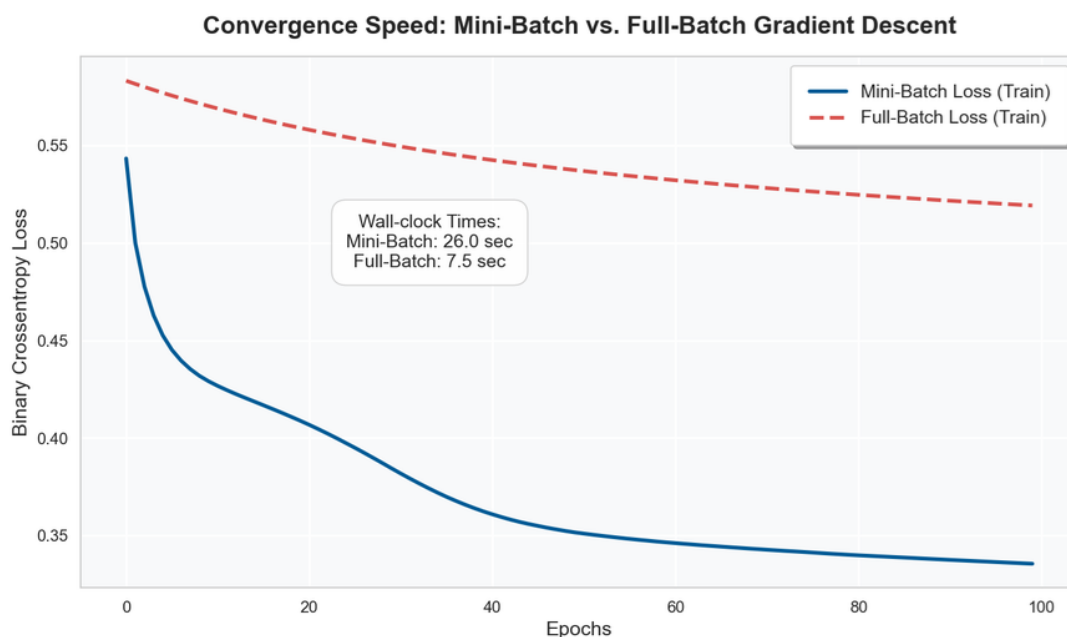


Figure 2: Loss curve comparison demonstrating the convergence behavior and wall-clock training times of Mini-Batch vs. Full-Batch Gradient Descent over 100 epochs.

6 Model Evaluation and Performance Metrics

Once the MLP model is trained, it is imperative to evaluate its generalization capability on the unseen test set. We utilize the Confusion Matrix, Classification Report, and the Area Under the Receiver Operating Characteristic Curve (ROC-AUC) to provide a holistic assessment.

6.1 Evaluation Implementation

The following Python code is utilized to generate the predictions on the test set and visualize the performance metrics using `seaborn` and `scikit-learn`.

```
1 import numpy as np
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 from sklearn.metrics import confusion_matrix,
    classification_report, roc_curve, auc
5
6 print("Evaluating the Mini-Batch Model on Test Data...")
7
8 # 1. Predictions
9 # Since output is sigmoid, we use 0.5 as threshold
10 y_pred_probs = model_mini.predict(X_test_scaled)
11 y_pred_classes = (y_pred_probs > 0.5).astype(int)
12
13 # 2. Classification Report
14 print("\n--- Classification Report ---")
15 print(classification_report(y_test, y_pred_classes))
16
17 # 3. Confusion Matrix Visualization
18 cm = confusion_matrix(y_test, y_pred_classes)
19 plt.figure(figsize=(8, 6), dpi=150)
20 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
21 xticklabels=['Retained (0)', 'Churned (1)'],
22 yticklabels=['Retained (0)', 'Churned (1)'],
23 annot_kws={"size": 14, "weight": "bold"})
24 plt.title('Confusion Matrix', fontsize=16, fontweight='bold', pad
    =15)
25 plt.xlabel('Predicted Label', fontsize=13, fontweight='medium')
26 plt.ylabel('True Label', fontsize=13, fontweight='medium')
27 plt.tight_layout()
28 plt.savefig('confusion_matrix.png', dpi=300)
29 plt.show()
30
31 # 4. ROC Curve & AUC Visualization
32 fpr, tpr, thresholds = roc_curve(y_test, y_pred_probs)
33 roc_auc = auc(fpr, tpr)
34
35 plt.figure(figsize=(8, 6), dpi=150)
36 plt.plot(fpr, tpr, color='#d9534f', linewidth=2.5, label=f'ROC
    curve (AUC = {roc_auc:.3f})')
37 plt.plot([0, 1], [0, 1], color='#005b96', linestyle='--',
    linewidth=2)
38 plt.xlim([0.0, 1.0])
39 plt.ylim([0.0, 1.05])
40 plt.xlabel('False Positive Rate (FPR)', fontsize=13, fontweight='
    medium')
41 plt.ylabel('True Positive Rate (TPR)', fontsize=13, fontweight='
    medium')
```

```

42 plt.title('Receiver Operating Characteristic (ROC)', fontsize=16,
    fontweight='bold', pad=15)
43 plt.legend(loc="lower right", fontsize=12, frameon=True, shadow=
    True)
44 plt.grid(color='gray', linestyle=':', linewidth=0.5, alpha=0.7)
45 plt.tight_layout()
46 plt.savefig('roc_curve.png', dpi=300)
47 plt.show()

```

Listing 11: Complete implementation of the MLP model, Training Strategies, and Visualization

6.2 Analysis of the Results

As depicted in Figure 3, the model's performance was evaluated comprehensively.

Confusion Matrix Analysis: The Confusion Matrix reveals that out of the total test samples, the model correctly identified 1543 retained customers (True Negatives) and 188 churned customers (True Positives). However, it misclassified 219 churned customers as retained (False Negatives). This behavior is highly characteristic of neural networks trained on imbalanced datasets, where the dominant class (0) inherently biases the decision boundary. Despite this, the model exhibits high precision for identifying stable customers.

ROC-AUC Analysis: To bypass the limitations of a rigid 0.5 threshold, the ROC curve was analyzed. The Area Under the Curve (*AUC*) reached a remarkable value of 0.863. In the context of behavioral finance and customer churn, an *AUC* > 0.85 indicates an excellent discriminative ability, proving that the non-linear mappings learned by the Multi-Layer Perceptron are highly effective.

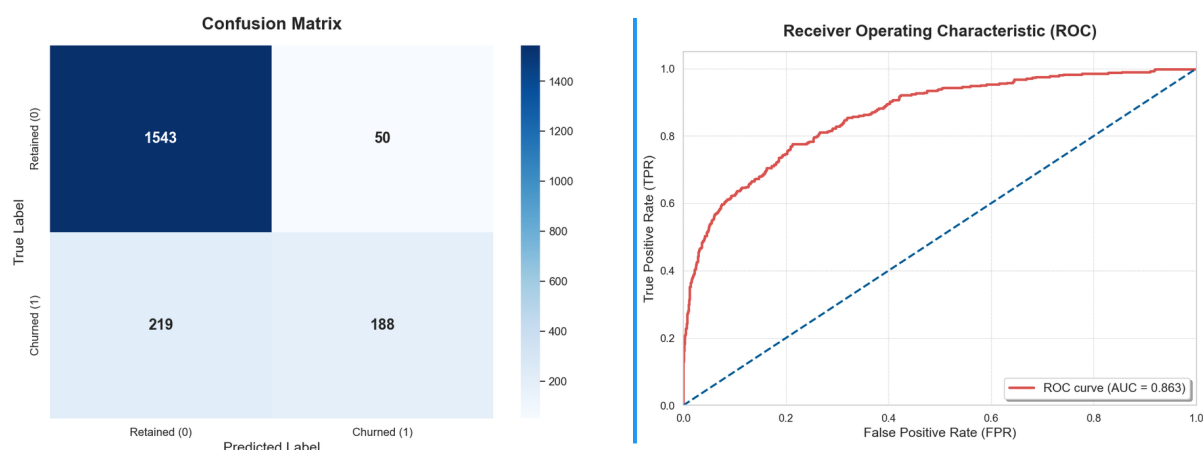


Figure 3: (Left) Confusion Matrix displaying the exact distribution of predictions. (Right) ROC Curve demonstrating an excellent AUC of 0.863.

7 Conclusion

The development and evaluation of the MLP model successfully demonstrate the power of deep learning in tabular data analysis. By comparing Full-Batch and Mini-Batch

training dynamics, we observed the trade-off between computational vectorization speed and convergence quality in the error space. The final Mini-Batch model, with an AUC of 0.863, stands as a robust predictive engine for bank management to proactively identify at-risk customers.